

AI Confessor: Real-Time Conversational AI Platform

Technical Project Report

ABSTRACT

AI Confessor is a voice-first real-time conversational AI web application built by Sai Koushik Vimmadisetty. A user speaks into the browser; microphone audio streams over a WebSocket to a FastAPI backend, where Vosk speech recognition running fully locally on the server transcribes it with no cloud call and no key, a large language model streams its reply token-by-token back over the same socket, and Edge TTS synthesizes the reply to MP3, which the browser plays through an audio queue. The stack is FastAPI (Python 3.12) on the backend, React + TypeScript + Vite on the frontend, and Docker Compose running the backend and the frontend behind nginx. LLM providers are pluggable (*anthropic / openai / mock*) and selected by the `LLM_PROVIDER` setting; the live deployment runs Anthropic Claude, model `claude-sonnet-4-6`. The system was deployed to AWS: an EC2 `t3.medium` instance in `us-east-1` running Ubuntu 24.04, Docker Compose (backend + frontend/nginx), free public HTTPS via a Cloudflare quick tunnel, and the Claude API key held server-side only. Quality assurance verified the implementation: 12/12 WebSocket protocol checks, 8/8 pytest backend tests, concurrent-session isolation, exact Vosk transcription of a test clip, and a clean secret scan. Live verification on AWS confirmed the HTTPS page load, a live (non-mock) Claude connection, a "PONG" reply test proving real model inference, and microphone capture with speech-to-text from an iPhone.

I. INTRODUCTION

Voice interaction with large language models is typically built on paid cloud speech APIs and request/response round trips that hide latency and add per-request cost. Developers who want a fully working, low-latency voice assistant face three frictions: (1) speech-to-text usually requires a paid API key, (2) model replies arrive only after full generation, hurting perceived responsiveness, and (3) multi-user deployments need isolated conversation state, which naive prototypes conflate. AI Confessor addresses these by keeping speech recognition local and free, streaming every generated token the moment it is produced, and isolating per-session histories behind asyncio locks.

Background and Objectives

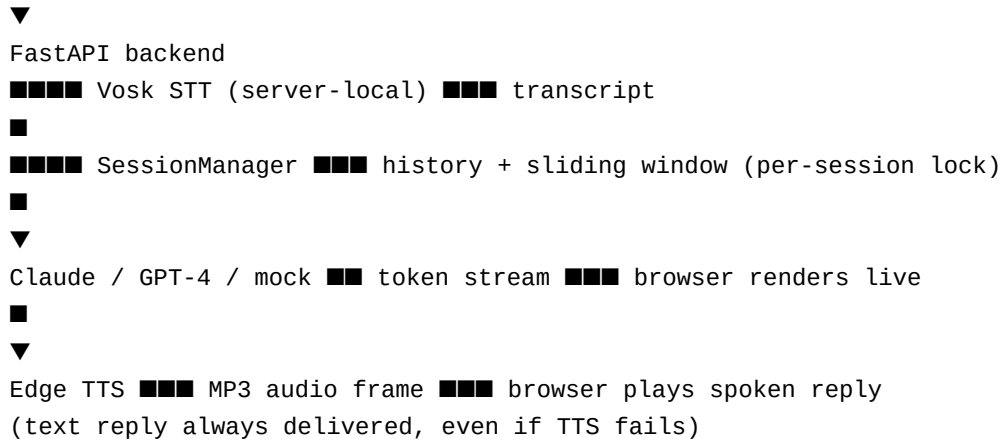
The project began as a portfolio-grade demonstration of real-time full-stack AI engineering: WebSocket streaming, speech processing, and LLM integration in one runnable system. The primary objectives were: a complete voice-in/voice-out loop with text fallback; a provider abstraction so the same codebase runs against Claude, GPT-4, or a keyless mock; per-session conversation memory that survives multi-turn dialogue; concurrent users with strictly isolated histories; a one-command Docker Compose run; no hardcoded secrets; and a real, publicly reachable AWS deployment. A secondary objective was process rigor: independent code review, runtime testing, a fix round, and re-verification before sign-off.

II. PROPOSED METHOD

The method centers on a single persistent WebSocket session per user that carries the full voice loop. The pipeline is: Mic → WebSocket → FastAPI → Vosk STT → LLM → Edge TTS → Speaker. Typed text turns take the same path minus the STT stage. Figure 1 shows the data flow for one voice turn.

Fig. 1 — Data flow (one voice turn)

Browser: mic / chat UI (React)
■ WebSocket /ws/chat – JSON frames



A. System Components

Each component below is listed with its responsibility and its dependencies, mirroring a schema-style dependency analysis.

TABLE I — System components

Component	Responsibility	Dependencies
backend/app/config.py	Env-driven settings: provider selection, model names, STT model, TTS voice, history window. No hardcoded secrets.	Environment variables only; LLM_PROVIDER empty triggers auto-select.
backend/app/llm.py	The <i>LLMClient</i> protocol (<code>stream_chat</code>) with Anthropic, OpenAI, and mock clients; factory <code>get_llm_client()</code> .	Config; official Anthropic/OpenAI SDKs; mock depends on nothing.
backend/app/sessions.py	Per-connection session registry; conversation history with a sliding window that always preserves the system prompt; per-session asyncio locks.	Config (system prompt, MAX_HISTORY_MESSAGES, default 20).
backend/app/stt.py	Vosk engine wrapper; auto-downloads <code>vosk-model-small-en-us-0.15</code> (~40 MB) on first run; consumes 16 kHz mono PCM chunks; emits partial and final transcripts.	Config (model name); local model files only.
backend/app/tts.py	Free keyless synthesis via <code>edge-tts</code> (default voice <code>en-US-AriaNeural</code>), returning MP3 bytes; raises a clean error the handler catches for text-only fallback.	Config (TTS_ENABLED, TTS_VOICE).
backend/app/main.py	The <code>/ws/chat</code> protocol; request orchestration (STT → history → LLM stream → TTS); REST endpoints <code>/health</code> and <code>/api/config</code> .	Sessions, LLM clients, STT, TTS, config.

React + TypeScript + Vite frontend	Chat window with live streaming render; microphone capture via AudioWorklet (16 kHz PCM); reply playback; a voice toggle that never disrupts the socket.	Backend WebSocket and REST endpoints only; no API keys in the client bundle.
Docker Compose + nginx	Separate backend/frontend containers; nginx serves the UI and proxies /api and /ws; one-command local run.	Docker host.
aws/deploy-ec2.sh	EC2 provisioning and Compose deployment automation (see Section III-G).	AWS credentials; Docker host.

III. TECHNIQUES USED

A. WebSocket Protocol

The /ws/chat endpoint exchanges typed JSON frames over one persistent connection per session: a session handshake on connect, token streaming of LLM replies, and typed frames for transcripts, audio, and control. Table II lists client-to-server messages; Table III lists server-to-client messages.

TABLE II — Client → Server

Type	Payload	Purpose
hello	—	Open session handshake
text	text	Submit a typed user turn
audio_chunk	base64 16 kHz/16-bit/mono PCM	Stream microphone audio
end_audio	—	Finalize utterance → final transcript
reset	—	Clear session history
ping	—	Keep-alive

TABLE III — Server → Client

Type	Payload	Purpose
session	session_id	Session identifier for this connection
transcript	text, final flag	Partial and final STT results
token	token	One streamed LLM token
audio	base64 MP3	Spoken reply (absent when TTS disabled/fails)
response_done	full_text	End of turn; complete reply text
reset_done	—	History cleared

pong	—	Keep-alive reply
error	message	Protocol error; connection stays alive

B. Session Isolation

Every WebSocket connection opens its own session with a unique session id, an independent conversation history, and a per-session asyncio lock. Histories use a sliding window (`MAX_HISTORY_MESSAGES`, default 20) that always preserves the system prompt, so multi-turn dialogue keeps context without unbounded growth. Concurrent sessions were verified to keep strictly separate histories: two connections receive distinct ids and neither sees the other's context.

C. Pluggable LLM Providers

Intelligence is abstracted behind the *LLMClient* protocol, with *AnthropicLLMClient*, *OpenAILLMClient*, and *MockLLMClient* implementations selected by the `LLM_PROVIDER` setting (`anthropic` / `openai` / `mock`). When the setting is empty, configuration auto-selects: an Anthropic key present selects `anthropic`, else an OpenAI key selects `openai`, else `mock`. The live AWS deployment runs `claude-sonnet-4-6`; mock mode exercises the entire pipeline with zero API keys.

D. Vosk Speech-to-Text

Speech recognition runs fully locally on the server via Vosk (model `vosk-model-small-en-us-0.15`), with no cloud call and no API key. The model (~40 MB) auto-downloads on first run; at inference time the engine consumes 16 kHz mono PCM chunks streamed as `audio_chunk` frames and emits partial and final transcripts. A 3.4 s English test clip was transcribed exactly ("the quick brown fox jumps over the lazy dog"), and live microphone capture with speech-to-text was confirmed from an iPhone against the AWS deployment.

E. Edge TTS Voice Replies

Replies are synthesized server-side to MP3 with `edge-tts` (default voice `en-US-AriaNeural`), a free keyless service. The MP3 bytes are streamed back over the WebSocket as an `audio` frame, which the browser plays through an audio queue; a voice toggle in the UI switches spoken replies on and off without touching the socket. The full reply text is always delivered in `response_done` even when synthesis is unavailable, so text fallback is guaranteed.

F. Security

The Claude API key lives server-side only in the backend `.env` (`chmod 600`); it is never embedded in frontend code, never appears in the client bundle, and never committed to Git. A secrets scan over the git history and the built frontend bundle found no key patterns, and the 8-test pytest suite includes explicit secret-leak checks.

G. AWS Deployment

The application was deployed to a live public URL on AWS — the centerpiece of this phase of the project. The host is an EC2 `t3.medium` instance in `us-east-1` running Ubuntu 24.04. The app runs in Docker Compose with the same two services as local development: the FastAPI backend and the frontend behind nginx, which serves the UI and proxies `/api` and `/ws`. Free public HTTPS is provided by a Cloudflare quick tunnel pointed at the frontend; HTTPS is required for phone browsers to grant microphone access. Deployment is automated with EC2 user-data, which launches the stack and publishes the tunnel URL through the frontend container so the live address is discoverable after boot. The project code is pushed to GitHub in the AI-Confessor repository. The live URL is

<https://passed-tampa-single-efforts.trycloudflare.com> — quick-tunnel URLs are ephemeral and change on redeploy, which is fine for a demo; a custom domain with a named tunnel would replace it for production.

IV. RESULTS

A. Quality Assurance

TABLE IV — Test results

Check	Result	Detail
WebSocket protocol	12/12 pass	Handshake, token streaming, multi-turn, ping/pong, reset, errors, voice loop
Backend test suite	8/8 pytest pass	Streaming, isolation, sliding window, concurrent sessions, secret-leak checks
Concurrent sessions	Pass	Distinct session ids; strictly independent histories
Vosk transcription	Pass	3.4 s English clip transcribed exactly
Secret scan	Pass	Git history and built frontend bundle clean of key patterns
Frontend build	Pass	Clean Vite production build; no key material in bundle

B. Live Verification on AWS

The deployment was verified end to end against the live URL. The page loads over HTTPS with no interstitial; the app header shows `claude-sonnet-4-6` connected rather than mock mode. A live chat test proved real model inference: instructed with a single word, the AI replied "PONG" exactly — a canned backend could not have produced that response, confirming the Anthropic Claude path is live. Microphone capture and speech-to-text were confirmed working from an iPhone over the HTTPS tunnel.

C. Honest Limitations

- The Cloudflare quick-tunnel URL is **ephemeral**: it changes on each redeploy. Acceptable for a demo; a custom domain plus a named tunnel is the production replacement.
- Edge TTS is a **network service**, so voice replies depend on connectivity to the synthesis endpoint; the text reply path is independent and always delivered.
- The deployment is a **single EC2 instance** with no high availability, no load balancing, and no managed database — appropriate for a demo, not for production traffic.
- No latency benchmarks are claimed; performance was not measured under controlled conditions.

V. SUMMARY

AI Confessor delivers a complete, live, voice-first conversational AI platform: FastAPI and WebSocket token streaming on the backend, a React voice/chat frontend, server-local Vosk speech recognition with no cloud key, Edge TTS voice replies with guaranteed text fallback, pluggable Claude/GPT-4/mock providers, and isolated per-session histories for concurrent users. Beyond the one-command Docker Compose local run, the system now runs publicly on AWS — an EC2 t3.medium in us-east-1 with free HTTPS through a Cloudflare quick tunnel, the API key held server-side only, deployment automated via EC2 user-data, and the code pushed to GitHub. Live verification confirmed HTTPS load, a live Claude connection (claude-sonnet-4-6), a "PONG" inference test, and iPhone microphone/STT capture. Remaining production work is explicit and bounded: a stable custom domain with a named tunnel, and high-availability hardening beyond the single instance.

REFERENCES

- [1] FastAPI — modern, fast web framework for building APIs with Python. <https://fastapi.tiangolo.com>
- [2] Vosk — offline open-source speech recognition toolkit. <https://alphacephei.com/vosk>
- [3] Anthropic — Claude large language models and the Messages API. <https://www.anthropic.com>
- [4] React — JavaScript library for user interfaces. <https://react.dev>
- [5] Docker — container platform; Compose for multi-container apps. <https://www.docker.com>
- [6] AWS EC2 — resizable compute capacity in the cloud. <https://aws.amazon.com/ec2>
- [7] Cloudflare Tunnel — secure outbound-only connectivity to Cloudflare's network.
<https://developers.cloudflare.com/cloudflare-one/connections/connect-networks>